

第1章 数据类型与变量

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 说出全部内建类型与字节数,并逐一找到它在 C 中的对应类型(或指出"没有对应")
- 掌握三种变量声明方式: `var`、短声明 `:=`、多重赋值,并理解**零值**机制——C 里"未初始化局部变量是 UB",Go 里**每个变量都有确定的初始值**
- 写得出各种字面量:二进制/八进制/十六进制、下划线分隔、复数、原始字符串
- 理解字符串是**不可变字节串**而非 `char[]:len()` 数的是字节不是字符, `rune` 才是"一个字符"
- 区分数组与切片:数组是**值类型**(与 C 数组退化为指针完全不同),切片是"指针+长度+容量"的视图
- 掌握显式类型转换:Go 没有隐式转换, `int` 与 `int64` 不能直接相加
- 能用 `iota` 写出类型安全的枚举常量,会用 `map` 完成键值存储

💡 从 C 到 Go:本章主题如何迁移

如果你会 C,读 Go 的第一感觉是"亲切":大括号、分号、`for`、`switch`、指针运算符 `&` 与 `*`,几乎原样搬来。但亲切之下藏着几处**根本性的哲学差异**,本章先把这些差异钉死,后面 13 章才能走得顺。

第一处差异是**安全性**:C 允许未初始化变量,读到垃圾值是未定义行为(UB);Go 给每个类型规定了**零值**——声明即初始化,你永远不会读到"随机数"。C 的数组名会退化成指针,传参时丢失长度;Go 的数组是完整复制,而真正的"动态数组"是切片,自带长度与容量。

第二处差异是**显式**:C 允许隐式类型转换(整型提升、`int` 转 `double` 自动发生),隐患埋在看不见的地方;Go 里一切转换都是显式函数调用, `int` 和 `int64` 被视为两种不兼容的类型,编译器直接拒绝混用。

第三处差异是**类型名称**:C 的 `int` 到底多大取决于平台(通常 32 位);Go 明确定义了 `int8 ... int64` 一整套固定宽度类型, `int` 在 64 位平台上就是 64 位,行为可预测。字符串在 C 里是 `char*` + 约定(以 `'\0'` 结尾),长度要自己 `strlen`;Go 的字符串是"指针+长度"的结构,内建 `len()` 直接给字节数,而且**不可修改**——这也意味着多字节字符(中文)的处理方式完全不同。

记住本章的一句话主线: C 把自由交给程序员(代价是陷阱),Go 把确定性交给程序员(代价是显式)。本章所有内容都可以用这枚"镜子"来对照。

📦 前置知识自查

- C 的基本类型: `char`、`short`、`int`、`long`、`float`、`double`、`_Bool` 的大小与取值范围
- 变量声明、初始化与"未初始化局部变量是 UB"的陷阱
- 字符与字符串:ASCII 码、`char[]` 与 `strlen`、转义序列
- 数组:下标访问、数组名退化为指针、`sizeof(arr)/sizeof(arr[0])` 的经典写法
- 结构体与指针的基本概念;一维数组的动态分配(`malloc`)

🚀 本章学习路线

1. **第一阶段**:通读 §1.1–§1.2,把类型表和零值表看熟,动手跑一遍"类型大小打印"示例
2. **第二阶段**:§1.3–§1.5,重点练字符串与 `rune` ——这是 C 程序员最容易踩的坑,建议把每个示例都亲手敲一遍
3. **第三阶段(实验)**:§1.6–§1.8 与章末实验题:学生成绩统计程序,强制覆盖全章类型
4. **验收标准**:能不看资料说出"10 个内建类型各自多大、零值是什么",能解释"为什么 `len("你好")` 是 6 而不是 2"

本章知识导图

- §1.1 从 C 到 Go:类型地图(内建类型表、与 C 类型——对照)
- §1.2 变量声明:`var`、`:=`、多重赋值与零值
- §1.3 字面量:整型/浮点/复数/字符/字符串(含原始字符串)
- §1.4 常量与 `iota`:从 C 的 `enum` 到 Go 的枚举惯用法
- §1.5 字符串:不可变的字节串(字节 vs 字符 vs `rune`)
- §1.6 数组与切片:值语义与视图(三要素 `len/cap/ptr`)
- §1.7 `map` 与枚举:键值存储与类型安全枚举
- §1.8 类型转换:一切显式,没有隐式

§1.1 从 C 到 Go:类型地图

Go 把所有内建类型按宽度明确命名。下表把 C 类型与 Go 类型并排列出——你会立刻发现:Go 消灭了" `int` 到底是几位"这类平台问题。

C 的写法	Go 的写法	字节数(64 位平台)	取值范围
<code>char</code>	<code>int8</code>	1	-128 ~ 127
<code>unsigned char</code>	<code>byte (= uint8)</code>	1	0 ~ 255
<code>short</code>	<code>int16</code>	2	-32768 ~ 32767
<code>int</code>	<code>int / int32</code>	8 / 4	平台相关;用 <code>int32</code> 则固定 4 字节
<code>long</code>	<code>int64</code>	8	$\pm 9.2 \times 10^{18}$
<code>float</code>	<code>float32</code>	4	约 6 位有效数字
<code>double</code>	<code>float64</code>	8	约 15 位有效数字
<code>_Bool</code>	<code>bool</code>	1	<code>true / false</code>
<code>wchar_t</code>	<code>rune (= int32)</code>	4	Unicode 码点(0 ~ 0x10FFFF)
<code>char*</code> 字符串	<code>string</code>	16(头)	不可变字节序列,任意长度

Go 独有而 C 没有的类型:

- **复数**: `complex64`、`complex128`,直接支持 `1+2i` 字面量与四则运算
- **无符号固定宽度族**: `uint8/16/32/64` 与 `uintptr` (存指针的整数,常用于底层编程)

- **错误值:** `error` 接口(第 4、9 章详解),替代 `errno` 全局变量

★ 重点:int 的大小是平台相关的,但 Go 给了你确定性

在 64 位平台上 `int` 是 64 位,在 32 位平台上是 32 位——与 C 的 `int` (几乎总是 32 位)不同。需要固定宽度时永远用 `int32` / `int64` 这类明确类型。协议字段、文件格式、哈希值一律用固定宽度类型,这是生产环境的铁律。

⚠ 易错点 1-1:以为 "int 一定是 32 位"

C 程序员的肌肉记忆。Go 在 64 位平台上 `int` 是 64 位,若你按 32 位假设做溢出判断、位运算或序列化,生产上会出隐蔽的 bug。规则很简单:拿不准宽度就用固定类型。

1.1.1 打印各类型大小

示例 1-1 用 `unsafe.Sizeof` 查看类型大小(64 位平台)

```
package main

import (
    "fmt"
    "unsafe"
)

func main() {
    var i8 int8
    var i int
    var i64 int64
    var f32 float32
    var f64 float64
    var c complex128
    var b bool
    var s string
    var p *int

    fmt.Println("int8   :", unsafe.Sizeof(i8), "字节")
    fmt.Println("int    :", unsafe.Sizeof(i), "字节") // 64 位平台为 8
    fmt.Println("int64  :", unsafe.Sizeof(i64), "字节")
    fmt.Println("float32:", unsafe.Sizeof(f32), "字节")
    fmt.Println("float64:", unsafe.Sizeof(f64), "字节")
    fmt.Println("complex128:", unsafe.Sizeof(c), "字节")
    fmt.Println("bool   :", unsafe.Sizeof(b), "字节")
    fmt.Println("string :", unsafe.Sizeof(s), "字节") // 16:指针 8 + 长度 8
    fmt.Println("指针   :", unsafe.Sizeof(p), "字节")
}
```

§1.2 变量声明:var、:= 与零值

C 的声明是"类型 + 名字",Go 保留了这种形式,同时提供了等价的**类型推断**写法。三种基本形态:

写法	含义	使用场景
<code>var n int</code>	声明变量,初始化为零值	需要零值开始的变量、包级变量
<code>var n int = 42</code>	声明并初始化	显式标注类型(如接口变量)
<code>n := 42</code>	短声明:推断类型	函数内部的最常用写法

★ 重点:零值——每个类型都有确定的默认值

类型	零值	C 的对照
数值类型(int / float / complex)	0	未初始化局部变量是 UB
bool	false	无真正布尔, _Bool 未初始化同样是 UB
string	"" (空串)	char* 可能是野指针
指针、切片、map、接口、channel	nil	NULL ;但 slice/map 有"nil 也能用"的特殊规则(第 14 章)
数组、结构体	逐元素零值	C 的局部数组不初始化是垃圾值

示例 1-2 零值:声明即初始化

```
package main

import "fmt"

func main() {
    var n int          // 0
    var f float64      // 0.0
    var ok bool        // false
    var s string       // ""(空串)
    var p *int          // nil(空指针)
    var arr [3]int      // [0 0 0]
    var sl []int        // nil 切片
    var m map[string]int // nil 映射

    fmt.Println(n, f, ok, s == "", p == nil, arr, sl == nil, m == nil)
}
```

示例 1-3 短声明 := 与多重赋值

```
package main

import "fmt"

func main() {
    x := 42                // 自动推断为 int
    name, age := "张三", 18
    name, age = "李四", 20 // 多重赋值,不需要中间变量

    a, b := 1, 2
    a, b = b, a            // 经典交换写法

    fmt.Println(x, name, age, a, b)
}
```

⚠ 易错点 1-2:短声明 "[:=" 要求至少一个新变量

`name, age := "张三", 18` 之后,再写 `name, age := "李四", 20` 会编译失败(没有新变量);正确写法是 `name, age = "李四", 20` (普通赋值)。但 `x, err := f()` 这种"一个新变量 + 复用旧变量"是合法的——`err` 已存在也没关系。

§1.3 字面量

Go 的字面量语法比 C 丰富:整型支持二进制、八进制(旧式 `0` 前缀和 `0o` 均可)、十六进制,以及 C23 才有的下划线分隔符(Go 从 1.13 起支持)。浮点支持科学计数法;复数直接写 `1+2i`。

示例 1-4 各种字面量

```
package main

import "fmt"

func main() {
    const bin = 0b1010    // 二进制:10
    const oct = 0o17       // 八进制:15
    const hex = 0x1F       // 十六进制:31
    const sep = 1_000_000  // 下划线分隔符:1000000
    const big = 3.14e10    // 科学计数法
    const cmplx = 1 + 2i   // 复数 1+2i

    const r = '中'         // 字符字面量:rune 类型
    const esc = "a\nb"     // 解释型字符串,支持转义
    const raw = `a\nb`     // 原始字符串:反斜杠原样保留

    fmt.Println(bin, oct, hex, sep, big, cmplx, r, esc, raw)
}
```

字符串字面量有两种:**解释型**(双引号,处理 `\n` `\t` 等转义,与 C 相同)与**原始型**(反引号,可跨行,反斜杠不转义)。原始字符串是写正则表达式、JSON 模板、SQL 语句的利器——C 里写正则要层层转义,Go 里直接粘贴。

§1.4 常量与 iota

C 的常量有 `#define` 与 `const` 两种, `#define` 是文本替换(不类型安全)。Go 只有一种 `const`, 但是**编译期常量**: 可以是数字、字符、字符串、布尔, 但不能是函数调用的结果。常量还有一个 C 没有的特性: 它们是**无类型常量**(untyped constant), 在赋值时按需要自动适配类型——所以 `const Pi = 3.14` 可以赋给 `float32` 也能赋给 `float64`。

示例 1-5 iota: 类型安全的枚举

```
package main

import "fmt"

// 一周七天:iota 从 0 开始自动递增
const (
    Sunday = iota // 0
    Monday         // 1
    Tuesday        // 2
    Wednesday      // 3
    Thursday       // 4
    Friday         // 5
    Saturday       // 6
)

// 位掩码:iota 与移位配合
const (
    Read = 1 << iota // 1
    Write             // 2
    Exec              // 4
)

func main() {
    fmt.Println(Sunday, Saturday) // 0 6
    fmt.Println(Read, Write, Exec)
}
```

对照 C 的 `enum`: Go 的 `iota` 没有"底层类型必须是 int"的限制, 可以枚举浮点、字符串; 而且每个值可以单独给出表达式。生产中最常见的组合是"iota + 类型别名": `type Color int` 之后再 `const ( Red Color = iota )`, 让枚举值带类型(第 14 章详述)。

§1.5 字符串: 不可变的字节串

这是 C 程序员本章最大的认知升级点。C 的字符串是 `char[]` + `'\0'` 约定: 可修改、长度靠扫描、中文按本地编码存字节。Go 的 `string` 是**不可变字节序列**: 内部是"指针 + 长度", 长度直接可得; 任何"修改"操作(拼接、替换)都产生新字符串, 原字符串不变。这带来两个直接推论:

- `len(s)` 得到的是**字节数**, 不是字符数; "你好" 的 `len` 是 6 而不是 2 (UTF-8 编码下每个汉字 3 字节)
- 下标 `s[i]` 取到的是一个**字节**(类型 `byte`), 不一定是完整字符

rune 是 Go 对"Unicode 字符"的表达: 本质是 `int32`, 存一个码点。用 `range` 遍历字符串时, Go 自动按 UTF-8 解码, 每次给出一个 **rune** 及其起始字节下标——这正是遍历中文的安全姿势。

示例 1-6 字符串是字节串: 字节 vs 字符 vs rune

```

package main

import (
    "fmt"
    "strings"
)

func main() {
    s := "你好,Go"
    fmt.Println(len(s)) // 9:两个汉字 6 字节 + 逗号 1 字节 + "Go" 2 字节

    // 下标访问得到的是字节
    fmt.Printf("%d %d\n", s[0], s[1]) // 228 189(UTF-8 首字节)

    // range 按 rune 遍历,正确处理多字节字符
    for i, r := range s {
        fmt.Printf("s[%d]=%q\n", i, r)
    }

    // 字符串是只读的:下面这行会编译失败
    // s[0] = 'h'

    // 常用操作(都在 strings 包)
    fmt.Println(strings.Contains(s, "Go"))
    fmt.Println(strings.HasPrefix(s, "你好"))
    fmt.Println(strings.Split(s, ","))
    fmt.Println(strings.ReplaceAll(s, "你", "您"))
    fmt.Println(strings.Join([]string{"a", "b", "c"}, "-"))
}

```

⚠ 易错点 1-3:用下标访问中文串,得到半个字符

"你好"[0] 的值是 228(你 的首字节),不是"你"这个字符。要对字符操作必须用 `range` 或先转 `[]rune`。同样, `len()` 用来数"字符个数"是经典 bug 源——统计字数请用 `utf8.RuneCountInString(s)`。

🚀 工程建议:string 与 []byte 互转有拷贝代价

`string([]byte{...})` 和 `[]byte(s)` 都会发生一次拷贝(为了维持"不可变"语义)。在热路径(如每秒处理几十万条消息)里反复互转会产生可观的分配压力。做法:能保持 `[]byte` 就一路用字节切片,只在边界处转换一次;必要时用 `strings.Builder` 拼接(第 5 章展开)。

§1.6 数组与切片:值语义与视图

C 里 `int a[10]` 传给函数时退化为 `int*`,长度信息丢失,所以 C 的函数总要额外传一个 `n`。Go 里 `[N]T` 是一个**完整的值**:赋值、传参都是整体拷贝,长度是类型的一部分(`[3]int` 与 `[4]int` 是不同类型)。日常开发中你几乎不会直接操作数组——真正的主角是**切片**(slice)。

C 的写法	Go 的写法
<code>int arr[10];</code> <code>sizeof(arr)/sizeof(arr[0])</code>	<code>arr := [10]int{}; len(arr)</code> (长度内建)
<code>void f(int *p, int n)</code> 手动传长度	<code>func f(s []int)</code> 切片自带长度
<code>malloc(n*sizeof(int))</code> + 自己管理 释放	<code>s := make([]int, n)</code> + GC 自动回收
数组名即指针,拷贝要 <code>memcpy</code>	数组赋值就是拷贝; <code>copy(dst, src)</code> 拷贝切片元素

切片的本质是一个三要素结构: 指针(指向底层数组)+ 长度(len)+ 容量(cap)。切片本身很小(24 字节),复制切片只是复制这三个数,底层数组共享——所以"切片是视图":对切片元素的修改,会反映到所有共享该底层数组的变量上。

示例 1-7 数组是值类型:传参即拷贝

```
package main

import "fmt"

func zero(a [3]int) {
    a[0] = 0 // 只修改副本
}

func zeroPtr(a *[3]int) {
    a[0] = 0 // 通过指针修改原数组
}

func main() {
    arr := [3]int{7, 8, 9}
    zero(arr)           // 传值:副本被改,原数组不变
    fmt.Println(arr) // [7 8 9]

    zeroPtr(&arr)      // 传指针:原数组被改
    fmt.Println(arr) // [0 8 9]

    // 数组可以直接赋值:整体拷贝
    b := arr
    b[1] = 100
    fmt.Println(arr, b) // [0 8 9] [0 100 9]
}
```

示例 1-8 切片:指针 + 长度 + 容量的视图


```

package main

import "fmt"

func main() {
    // make 创建:长度 3,容量 5
    s := make([]int, 3, 5)
    fmt.Println(s, len(s), cap(s)) // [0 0 0] 3 5

    // 追加:容量足够时直接写底层数组
    s = append(s, 1, 2)
    fmt.Println(len(s), cap(s)) // 5 5

    // 切片是视图:改元素会穿透到底层数组
    arr := [4]int{1, 2, 3, 4}
    v := arr[1:3]
    v[0] = 20
    fmt.Println(arr) // [1 20 3 4]

    // copy:真正复制元素
    dst := make([]int, len(v))
    copy(dst, v)
    dst[0] = 99
    fmt.Println(v, dst) // [20 3] [99 3]
}

```

⚠ 易错点 1-4:append 后共享底层数组的"幽灵修改"

两个切片共享底层数组时,一个 `append` 可能悄悄改写另一个的内容——特别是当容量还够时, `append` 不分配新数组,直接写进共享区。生产建议:拿不准就显式 `copy` 一份再操作;不要依赖"append 一定会拷贝"的直觉。

§1.7 map 与枚举

C 标准库没有哈希表(要自己实现或引入第三方)。Go 内建 `map[K]V`,K 必须是可比较类型(数值、字符串、指针、数组、结构体;切片和 map 不行)。读不存在的键返回零值,并用第二个返回值报告"键是否存在"——这是 Go 最常见的惯用法之一。

示例 1-9 map 基本操作

```

package main

import "fmt"

func main() {
    // 声明并初始化
    scores := map[string]int{"张三": 90, "李四": 85}

    // 读取:第二个返回值表示键是否存在
    s, ok := scores["王五"]
    fmt.Println(s, ok) // 0 false

    // 写入与删除
    scores["王五"] = 88
    delete(scores, "李四")

    // 遍历(顺序随机,不要依赖顺序)
    for name, score := range scores {
        fmt.Println(name, score)
    }

    // 用 make 预分配容量,避免多次扩容
    big := make(map[string]int, 10000)
    fmt.Println(len(big))
}

```

§1.8 类型转换:一切显式

C 的隐式转换(整型提升、`double x = 1/2` 得 0.5 还是 0?答案是 0)是 bug 温床。Go 的规则简单粗暴:没有隐式数值转换。`int` 与 `int64` 相加、`float64` 赋给 `float32` ,统统编译错误。转换就是一次显式调用: `T(v)` 。

示例 1-10 显式转换

```

package main

import "fmt"

func main() {
    var n int = 42
    var f float64 = float64(n) // int → float64
    var i int32 = int32(f)      // float64 → int32(截断小数)

    // 字符串与字节/字符切片互转
    s := string([]byte{'h', 'i'})
    bytes := []byte(s)
    runes := []rune("中文") // 按字符转,每个 rune 4 字节
    back := string(runes)

    fmt.Println(f, i, s, string(bytes), len(runes), back)

    // 下面的代码全部编译失败(Go 无隐式转换)
    // var x int64 = n          // int → int64 必须显式
    // f = 1 + f                // 字面量 1 是无类型常量,可以
    // d := 1.0
    // f = f + d                // float64 与 float32 不能混用
}

```



工程建议:转换前想清楚精度与溢出

`float64` → `int` 会截断小数; `int` → `int8` 超出范围会回绕(Go 的转换是定义良好的截断,不会 panic)。在涉及协议解析、金额计算时,先做范围检查再转换,并用固定宽度类型;涉及单位换算(字节→KB)时用 `float64` 中间量避免整数除法截断。

本章速查表

项目	要点
整型族	int8/16/32/64、uint8/16/32/64; int 平台相关(64 位平台为 8 字节)
别名	byte = uint8; rune = int32 (Unicode 码点)
浮点/复数	float32、float64、complex64、complex128
布尔	bool, 只有 true / false, 不与整数互转
字符串	不可变字节串; len() 是字节数; 下标取字节; range 得 rune
声明语法	var n int; var n int = 1; n := 1 (函数内)
零值	数值 0、布尔 false、字符串 ""、其余 nil; 无"未初始化"
多重赋值	a, b = b, a 交换; := 要求至少一个新变量
字面量	0b1010 0o17 0x1F 1_000_000 1+2i
字符串字面量	双引号(转义)与反引号(原始、可跨行)
常量	const 编译期常量; iota 枚举递增器
数组	值类型, 整体拷贝; [3]int 与 [4]int 不同型
切片	三要素: 指针+len+cap; make / append / copy; 视图共享底层数组
map	map[K]V; 读缺键得零值; v, ok := m[k] 判断存在
类型转换	只有显式 T(v); 无隐式转换; int 与 int64 不互容

最小必会清单

- 能不看资料说出全部内建数值类型及其字节数, 说出 int 与 int32 的区别
- 能说出五个"零值": 数值、布尔、字符串、指针、切片的零值分别是什么
- 能解释 len("你好") == 6 的原因, 并写出按字符遍历字符串的正确写法
- 能说清数组与切片的三点区别: 拷贝语义、长度是否属于类型、传参是否丢长度
- 能写出 append、copy、map 增删查的完整代码
- 能解释"Go 没有隐式类型转换"的含义, 并写出 int → float64 → int32 的转换链
- 能写出 iota 枚举与位掩码常量
- 能区分解释型字符串与原始字符串的使用场景

自测题

Q 1. 在 64 位平台上, int、int32、int64、byte、rune 分别占几个字节?

✓ 参考答案

8、4、8、1、4。`int` 与平台相关(64 位平台 8 字节); `byte` 是 `uint8` 的别名; `rune` 是 `int32` 的别名,存一个 Unicode 码点。

Q 2. 下列变量声明哪些有编译错误?为什么?(a) `var s string`;(b) `s := "a"; s = "b"`;(c) `s := "a"; s[0] = 'b'`;(d) `x := 1; x, y := 2, 3`

✓ 参考答案

(a) 合法, `s` 的零值是 `""`;(b) 合法, `=` 是赋值,字符串变量本身可变(只是字符串内容不可变);(c) 编译错误,字符串内容不可变,不能下标赋值;(d) 合法, `:=` 中 `y` 是新变量,允许复用 `x`。

Q 3. 写出 `s := "Hello, 世界"` 的 `len(s)` 值,并说明 `for i, r := range s` 中 `i` 与 `r` 的含义。

✓ 参考答案

`len(s) = 13`(ASCII 部分 7 字节 + 逗号 1 字节 + 两个汉字 6 字节)。`i` 是每个 `rune` 的起始字节下标, `r` 是 `rune` 本身。遍历到 `r = '世'` 时 `i = 8`(前 7 字节是 "Hello,",加上空格或标点按实际字节数递增)。

Q 4. `a := [3]int{1,2,3}` 与 `b := a` 后执行 `b[0] = 99`, `a[0]` 是多少?若把数组换成切片,结果又如何?为什么?

✓ 参考答案

数组:仍是 1, `b := a` 是整体拷贝。切片: `a[0]` 变成 99,因为切片复制只复制"指针+len+cap",两个切片共享底层数组,改元素互相可见。

Q 5. 下面代码输出什么? `n := 3.9; m := int(n); x := 5; y := x / 2`

✓ 参考答案

`m = 3` (浮点转整型截断小数); `y = 2` (整数除法,与 C 相同)。若想要 2.5,应写 `float64(x) / 2`。

Q 6. `m := map[string]int{}` 与 `var m map[string]int` 有什么区别?对两者执行 `m["a"] = 1` 会怎样?

✓ 参考答案

前者是空 `map`(可写),后者是 `nil map`。`m["a"] = 1` 对空 `map` 正常;对 `nil map` 会 **panic**(运行时不存在的内存位置写入)。读 `nil map` 是安全的(返回零值),写会崩溃——这是第 14 章"nil 的陷阱"的预告。

章末实验题:学生信息登记与成绩统计

实验 1:学生信息登记 + 成绩统计

实验目标:写一个命令行程序,登记学生(姓名、性别、数学成绩、英语成绩),并输出成绩单与统计结果,覆盖全章类型与语法。

实验要求:

- 任务 1:用结构体 + 数组定义"学生"模型,数学/英语成绩用 `[CourseCount]float64` 定长数组保存(覆盖:结构体、数组、常量)
- 任务 2:用 `iota` 定义课程枚举(数学/英语),并用它做数组下标(覆盖:`iota` 枚举、数组下标)
- 任务 3:从标准输入读入一名新学生,姓名是字符串,性别是布尔,成绩是浮点(覆盖:字符串、布尔、浮点、格式化读入)
- 任务 4:统计每门课的最高/最低/平均分与及格人数,注意 `len()` 返回 `int`、除法需要显式转换(覆盖:显式类型转换、切片遍历)
- 任务 5:用 `fmt.Printf` 输出对齐的成绩单(覆盖:格式化输出、字符串处理)
- 任务 6:追加学生时验证 `append` 与切片的关系(覆盖:切片、`append`)

实验环境:Go 1.21+, `go run main.go` 运行, `go vet` 检查代码。

第一步 定义常量和枚举: `const passLine = 60` 与 `CourseMath = iota` 等,想清楚为什么用 `iota` 而不用魔法数字 0、1

第二步 定义 `student` 结构体,成绩字段用定长数组而非切片——体会"数组是值类型"带来的确定性

第三步 `calcStat` 函数,遍历学生切片求统计值;注意平均分计算必须先转 `float64`

第四步 主流程:预置三名学生,再从标准输入读入第四名,最后打印成绩单与统计结果

✔ 参考答案要点

```

// 学生信息登记 + 成绩统计
package main

import (
    "bufio"
    "fmt"
    "os"
    "strconv"
    "strings"
)

// 常量:及格线(常量知识点)
const passLine = 60

// iota 枚举课程:第 2 个值也用于数组长度(枚举知识点)
const (
    CourseMath = iota // 数学
    CourseEnglish      // 英语
    CourseCount        // 课程总数
)

// 学生模型:姓名 + 性别 + 定长成绩数组(结构体/数组/布尔)
type student struct {
    name  string
    male  bool
    scores [CourseCount]float64
}

// 统计结果
type stat struct {
    max, min, avg float64
    passers      int
}

// calcStat 统计某门课(遍历切片 + 显式转换)
func calcStat(list []student, course int) stat {
    var s stat
    s.min = 100
    total := 0.0
    for _, stu := range list {
        sc := stu.scores[course]
        if sc > s.max {
            s.max = sc
        }
        if sc < s.min {
            s.min = sc
        }
        total += sc
        if sc >= passLine {
            s.passers++
        }
    }
    if len(list) > 0 {
        s.avg = total / float64(len(list)) // int → float64 显式转换
    }
}

```



```

return s
}

func main() {
    students := []student{
        {name: "张三", male: true, scores: [CourseCount]float64{92, 78}},
        {name: "李四", male: false, scores: [CourseCount]float64{55, 88}},
        {name: "王五", male: true, scores: [CourseCount]float64{71, 65}},
    }

    // 追加新学生:字符串/字符/格式化读入
    fmt.Print("输入新学生:姓名 性别(m/f) 数学 英语,空格分隔: ")
    reader := bufio.NewReader(os.Stdin)
    line, _ := reader.ReadString('\n')
    fields := strings.Fields(line)
    if len(fields) == 4 {
        math, err1 := strconv.ParseFloat(fields[2], 64)
        eng, err2 := strconv.ParseFloat(fields[3], 64)
        if err1 == nil && err2 == nil {
            students = append(students, student{ // append 扩展切片
                name:   fields[0],
                male:   fields[1] == "m",
                scores: [CourseCount]float64{math, eng},
            })
        }
    }

    // 打印成绩单(格式化输出)
    fmt.Printf("%-6s %-4s %8s %8s\n", "姓名", "性别", "数学", "英语")
    for _, stu := range students {
        gender := "女"
        if stu.male {
            gender = "男"
        }
        fmt.Printf("%-6s %-4s %8.1f %8.1f\n",
            stu.name, gender, stu.scores[CourseMath], stu.scores[CourseEnglish])
    }

    // 统计输出
    m := calcStat(students, CourseMath)
    e := calcStat(students, CourseEnglish)
    fmt.Printf("数学:最高 %.1f 最低 %.1f 平均 %.1f 及格 %d 人\n",
        m.max, m.min, m.avg, m.passers)
    fmt.Printf("英语:最高 %.1f 最低 %.1f 平均 %.1f 及格 %d 人\n",
        e.max, e.min, e.avg, e.passers)
}

```

设计说明:成绩用定长数组并让 `CourseCount` 参与数组长度,新增课程时只改常量一处;统计函数通过切片遍历,依赖"数组是值类型"保证每个学生数据独立;性别用布尔避免 `'m'/'f'` 魔法字符;所有数值计算都在显式类型规则内完成,编译期即可保证不混用类型。

下一章预告:第2章 运算符与表达式。本章学会了"数据长什么样",下一章学习"数据怎么运算":Go 的算术溢出行为、没有三目运算符、`++` 不是表达式、位运算的 `&^` 特性——每一条都与 C 的习惯形成对照。